

LISTS

Unit 5

Presented By: Saroj Bhandari

Syllabus

- Introduction
- Static and Dynamic List Structure,
- Array Implementation of Lists,
- Queue as a list

Introduction to List

- In the **Data Structures and Algorithms (DSA)**, a "list" refers to a fundamental data structure that stores a collection of **elements in a specific order**, allowing for dynamic size and potential duplicates, and where each element can be accessed using its index.
- **Characteristics of Lists**
 - **Ordered**: Elements maintain a specific sequence.
 - **Duplicates allowed**: A list can have repeated elements.
 - **Dynamic size**: Depending on the type of list, it may grow or shrink in size.
 - **Indexed**: Each element has an index (position) associated with it.

3. Types of Lists

a. Array List (Static List)

- Fixed size (declared beforehand).
- Direct access to elements using an index.
- Faster access but resizing is costly.

b. Dynamic List

- Can grow or shrink as needed.
- Commonly implemented using arrays or linked structures.
- Examples: Python lists, C++ vectors.

Type of List

c. Circular List

- The last element connects to the first element, forming a circle.
- Useful for applications like round-robin scheduling.

d. Doubly Circular List

- Elements are connected in both directions (forward and backward).
- The last element connects to the first and vice versa.

Array Implementation of List

- The **array implementation of a list** means using an array to store and manage list elements sequentially in memory.
- The **array implementation** of a list involves representing the list as an array and writing functions to handle list operations.

Operations in **Array Based List**

Key Operations in Array-Based Lists:

- **Creation** — Define an array of a certain size.
- **Insertion** — Add elements at any position by shifting others.
- **Deletion** — Remove elements and shift others left.
- **Modification** — Change an element's value by accessing its index.
- **Traversal** — Loop through all elements to display or process them.
- **Merging** — Combine two lists into one array.

1. Creating the List

Algorithm:

1. Define an array of size N
2. Insert initial element into the array
3. Print the array

Example:

```
List:  [10, 20, 30, 40, 50]  
Index:  0  1  2  3  4
```


Creating the List...

Code

```
#include <stdio.h>

int main() {
    int list[5] = {10, 20, 30, 40, 50};
    printf("Initial List: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", list[i]);
    }
    return 0;
}
```

2. Inserting a New Element

Algorithm:

- Shift elements from the target position to the right.
- Place the new element at the desired position.
- Increase the list size.

Example: Lets insert 25 at index 2

```
Before: [10, 20, 30, 40, 50]
After:  [10, 20, 25, 30, 40, 50]
Index:  0  1  2  3  4  5
```

Inserting Element (Code)

```
#include <stdio.h>
int main() {
    int list[6] = {10, 20, 30, 40, 50};
    int n = 5; // Current size
    int pos = 2, newElement = 25;
    // Shift elements to the right
    for (int i = n; i > pos; i--) {
        list[i] = list[i - 1];
    }
    // Insert the new element
    list[pos] = newElement;
    n++;
    printf("After Insertion: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", list[i]);
    }
    return 0;
}
```

3. Deleting an Element

Algorithm:

- Find the position to delete.
 - Shift elements to the left to fill the gap.
 - Decrease the list size.
- Example:
 - Let's delete the element at Index 3

Before: [10, 20, 25, 30, 40, 50]

After: [10, 20, 25, 40, 50]

Index: 0 1 2 3 4

Deleting an Element (Code)

```
#include <stdio.h>

int main() {
    int list[6] = {10, 20, 25, 30, 40, 50};
    int n = 6, pos = 3;
    // Shift elements to the left
    for (int i = pos; i < n - 1; i++) {
        list[i] = list[i + 1];
    }
    n--;

    printf("After Deletion: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", list[i]);
    }
    return 0;
}
```

4. **Modifying** an Element

- **Algorithm:**

- Find the index of the element.
- Replace it with the new value.

- **Example:**

- Let's change the element at index 1 from 20 to 23

```
Before: [10, 20, 25, 40, 50]
```

```
After:  [10, 22, 25, 40, 50]
```

```
Index:  0  1  2  3  4
```

Modifying an Element(Code)

```
#include <stdio.h>

int main() {
    int list[5] = {10, 20, 25, 40, 50};
    // Modify the element at index 1
    list[1] = 22;
    printf("After Modification: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", list[i]);
    }
    return 0;
}
```

5. Traversing the List

Algorithm:

- Loop through each index of the list.
- Display each element.

- **Example:**

List Elements:

Index 0 → 10

Index 1 → 22

Index 2 → 25

Index 3 → 40

Index 4 → 50

Traversing List (Code)

```
#include <stdio.h>

int main() {
    int list[5] = {10, 22, 25, 40, 50};
    printf("Traversing the List:\n");
    for (int i = 0; i < 5; i++) {
        printf("Element at index %d: %d\n", i, list[i]);
    }
    return 0;
}
```

6. Merging Two Lists

Algorithm:

- Copy elements from the first list.
- Append elements from the second list.
- Output the merged list.

Example

```
Array1: [10, 22, 25, 40, 50]
```

```
Array2: [60, 70, 80]
```

```
Merged: [10, 22, 25, 40, 50, 60, 70, 80]
```

Merging two lists(Code)

```
#include <stdio.h>
int main() {
    int list1[5] = {10, 22, 25, 40, 50};
    int list2[3] = {60, 70, 80};
    int merged[8], i;
    // Copy elements from list1
    for (i = 0; i < 5; i++) {
        merged[i] = list1[i];
    }
    // Append elements from list2
    for (int j = 0; j < 3; j++, i++) {
        merged[i] = list2[j];
    }
    printf("Merged List: ");
    for (i = 0; i < 8; i++) {
        printf("%d ", merged[i]);
    }
    return 0;
}
```

Queue as a list

- Implementation of linear Queue
 - Covered in previous chapter

- **Revision**

```
Queue: [ _ , _ , _ , _ , _ ]  
Front: -1  
Rear:  -1
```

```
Enqueue(10):  
Queue: [10, _ , _ , _ , _ ]  
Front:  0  
Rear:   0
```

```
Enqueue(20):  
Queue: [10, 20, _ , _ , _ ]  
Front:  0  
Rear:   1
```

```
Enqueue(30):  
Queue: [10, 20, 30, _ , _ ]  
Front:  0  
Rear:   2
```

```
Dequeue():  
Queue: [_, 20, 30, _ , _ ]  
Front:  1  
Rear:   2
```

```
Dequeue():  
Queue: [_, _, 30, _ , _ ]  
Front:  2  
Rear:   2
```

```
Peek(): 30 (Front element)  
Queue: [_, _, 30, _ , _ ]
```

```
IsEmpty(): NO (Front = 2, Rear = 2)
```

5. Advantages of Lists

- Easy to implement and use.
- Supports **dynamic resizing (for dynamic lists)**.
- Allows quick access to elements via indexing.
- Flexible to store different data types **(in languages like Python)**.

6. Disadvantages of Lists

- **Insertion and deletion** can be slow for large lists (shifting elements is needed).
- Static lists **waste memory** if declared larger than needed.
- Searching is **inefficient in unsorted lists**.

8. Applications of Lists

- **Storing data** (e.g., student names, numbers).
- **Implementing other data structures** (e.g., stacks, queues).
- **Database indexing.**
- **Processing jobs** (e.g., CPU scheduling).



Thank you !!!